CS554-Final Project

Joseph Wiseman, David Schwartz

December 2025

1 Introduction

Pekko [2], is a Scala [13] library that implements the actor concurrency model. The actor model was first introduced in 1973 [6]. In the model, a program consists of many actors which encapsulate small, mostly independent computing tasks. Actors use message passing to communicate with one another. Each actor maintains a mailbox to buffer messages. As any actor receives a message to their mailbox, they run their computation and can send out resultant messages. Pure actor-based systems use actors for everything—even primitives are considered actors. More commonly, as is the case for Pekko, actors are objects.

Traditionally, actors are implemented at the language level; several actor languages are Planner [5], Act [1], Newspeak (Smalltalk-esque language) [3], and most famously Erlang [4]. Those languages must ensure correct performance given the hardware (memory model) on which the program is being deployed, or at least give some guarantees so as to inform developers that wish to use those languages. However, Pekko is not a language—it is a library for Scala, a hybrid functional language that runs on the Java Virtual Machine (JVM) [9]. Due to Scala being a hybrid language, there can be side effects, and developers can use lower level concurrency primitives. The library gives specific guarantees, with the caveat that users do not "mix paradigms" and use Java Memory Model (JMM) [11] concurrent primitives.

We investigated how Pekko uses concurrency primitives of the JMM to implement actors and under which scenarios can programmers unintentionally break the model. We found that the `Mailbox` within Pekko is a crucial data structure which provides the majority of the guarantees needed for an actor system. In addition, the happens-before relationships inside of the `Mailbox` are unclear as the developers mix atomic and essentially non-atomic operations on the same address. However, the implementation is still correct, as it exploits properties of the JMM (see Section 3.3 for details). Regardless, programmers can still easily break Pekko and the JVM by creating actors which rely on other concurrency primitives (see Section 4.1 for an example concerning locks).

2 Background

Before going further into the correctness of Pekko's implementation, some background on both actor systems and the JMM is required.

2.1 Actor Based Concurrency

Actors are a model for highly concurrent programming. Actors can perform three basic actions: 1) send messages to other actors, 2) create new actors, and 3) change the behavior used to process the next message. Message passing between actors is "a universal control primitive" [6]. Actors have mailboxes to buffer messages that are sent. Messages themselves are unidirectional. An actor is neither expected nor required to respond to any message it receives. Consequently, all actor communication is asynchronous, leading to programs which can easily be parallelized, as many opportunities for deadlocking (caused via blocking calls) are eliminated. Due to actors being isolated, other than the messages that they receive, if an actor fails for any reason, the rest of the system of actors may still operate without any change. This point alludes to a core philosophy in the Erlang language, "Let it crash.[4]" Actors can be restarted or respawned when absolutely needed.

A simple example actor, presented in [6], is a `Cell`, which guards a single piece of memory. The `Cell` actor has exclusive access to the memory it protects. Other actors can send `get` and `set` messages. The `Cell` responds to a `get` message—sending the value at the memory address. The `set` message includes a value to write to the protected address. Given this context, **the mailbox determines the modification order of memory protected by the `Cell`.** Understanding the function of the mailbox will be critical to our understanding of Pekko.

Pure actor systems model all aspects of the language through actors—even control flow and types that are primitives (e.g., integers, strings, etc) in other languages [6, 5]. Pure actor systems are designed to be functional in that actors' behaviors are not stateful; albeit the actor implicitly has some state because of the mailbox. To represent state changes, actors either have to send the state via a message or spawn a replacement actor which contains an immutable copy of the updated state (similar to cloning data in a functional language). More commonly, actors are treated like objects (e.g., in Pekko and Erlang) and thus can have some internal state. Being "object-like" is good for Scala and Pekko as the JVM is object-oriented. However, message passing (in the way that actors use it) is not a concurrency primitive in the JVM (or consequently in Scala), so Pekko must implement it.

2.2 Java Memory Model

Scala is a programming language which targets the JVM. As a result, it is subject to the behavior of the JMM. According to the JMM, there are 4 general rules (described in plain text) to indicate a happens-before relationship:

1. if 2 actions are in program order within the same thread
2. objects are guaranteed to have a happens-before edge from the end of the constructor to the start of the finalizer
3. if 2 actions have a synchronizes-with relationship
4. happens-before is transitive

Furthermore, there are several actions that induce a synchronizes-with relationship in the JMM, but only two are important for our context:

1. unlock action on a monitor (essentially a per-object lock) synchronizes-with all subsequent lock actions on the same monitor

2. writes to volatile variables synchronizes-with all subsequent reads of the same variable

It is also important to note that the source and destination of any synchronizes-with edge has release-acquire semantics.

In addition, the JVM provides several mechanisms to access read-modify-write instructions, which are useful for enforcing correct concurrent behavior—i.e., as they are necessary to implement locks. *Atomic* data structures (e.g., `AtomicInteger` and `AtomicReference`) provide compare-and-swap (CAS) behavior over their respective data types. In Java 9+, `VarHandle` [15] was introduced, which allows developers to specify the memory order semantics with `get`, `set`, and `CAS` operations on fields. Similarly to the C/C++ 11 memory model [7], different functions in `VarHandle` provide *sequentially consistent*, *release*, *acquire*, *relaxed*, and *plain* semantics. The documentation even references the C/C++ 11 memory model. Plain semantics means non-volatile read/write, so no happens-before relationships exist between conflicting accesses in different threads; we will revisit this later in Section 3.3. In [10], plain accesses are described to be similar to non-atomic memory in C/C++ 11. Pekko uses a combination of these techniques to ensure a correct implementation of the actor model [12].

3 Pekko

3.1 Happens-Before Guarantees

Pekko describes two happens-before relationships (in plain text) in its documentation [12].

1. The send rule: the send of the message to an actor happens before the receive of that message by the same actor.

2. The subsequent processing rule: processing of one message happens before processing of the next message by the same actor.

An important observation is that actors are not necessarily thread-bound—the default implementation lets them move between threads based on availability. Based on this observation, Pekko needs to provide happens-before relationships between message processing in order to allow the state of actors to correctly move across threads. Without this relationship, a developer would need to mark all fields within an actor as volatile, as that would manually insert the necessary synchronizes-with behavior. This action could also reduce the performance of the system. The implications of this property, in the context of the JVM, are explored in more detail in Section 4. Before discussing how Pekko provides those guarantees, we will discuss the key abstractions in Pekko's source code.

3.2 Abstractions

The Pekko codebase is expansive due to its highly configurable nature. Various pieces of the library can be fine-tuned to the user's desired machine, application, or server use case. This means that at times the code can get very dense. Furthermore, Pekko uses reflection to fill in configured code at runtime. In order to limit our scope, we decided to only investigate the default configurations.

To determine how Pekko works, we wrote a sample program (discussed more in Section 4) and used a debugger to manually trace the system. Figure 1 shows our observations on key abstractions in Pekko.

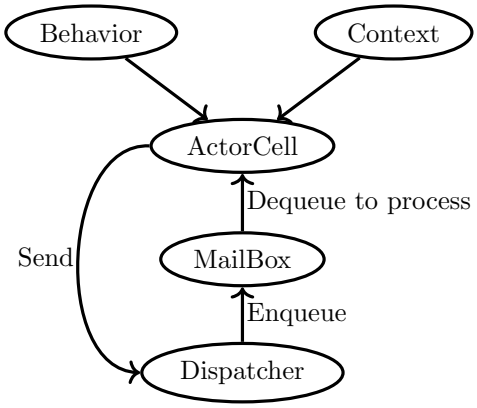

Figure 1: Information flow

The center point of Pekko is the `Dispatcher`, which handles scheduling actors for execution and sending messages from one actor to another. An odd part of the code is that the `Dispatcher` does not schedule actors themselves, but rather schedules their `Mailbox`es. The `Mailbox` is both an actor-specific queue of messages to process and a `Runnable`—a function containing the code which dequeues messages and invokes the actor to process them appropriately.

Scheduling is done via an `ExecutorService` inside of the `Dispatcher`. An `ExecutorService` is Java's implementation of a thread pool. This brings up an interesting distinction between Pekko and Java. In Java, a thread pool runs an individual task (e.g., a `Runnable`) on a single thread in the pool *until it completes*. Pekko's `Mailboxes` are configured to exit their task early if either: 1) the `Mailbox` is empty or 2) processing has exceeded some preconfigured execution time threshold. Afterward, the actor and `Mailbox` remain in the system, but need to be rescheduled.

An `ActorCell` encapsulates an Actor. The cell is the combination of a `Behavior`, which in essence is a function/closure on how to process messages, and the execution context, which allows the actor to send messages to others. A `Behavior` returns another `Behavior` holding the function to process the next message, and so on, until the `Mailbox` is empty or runs out of execution time.

3.3 Correct Concurrency

To be thread-safe, Pekko must protect memory accesses in the `Dispatcher` and, more importantly, the `Mailbox`. Both classes make frequent use of volatile semantics to provide happens-before relationships over fields. The default `Mailbox` is a multi-producer, single-consumer queue (based on [16]) which is implemented via linked nodes.

The queue enqueues at the head and dequeues from the tail. The head of the queue is an `AtomicReference`. During enqueues, the head undergoes a `get` and `set` operation (similar to a CAS, but no expected value is provided) on the head, which involves a read and write, both with volatile semantics; therefore, a happens-before relationship exists.

The tail of the queue is referenced via a `VarHandle`. Updating the tail uses *release* semantics. However, reading the tail node uses a *plain non-volatile* `get` operation. Reading the source code, it is not clear that a happens-before relationship exists here; however, we were unable to find any evidence of the queue failing or having an inconsistent state.

An article by Doug Lea [10] briefly mentions that this pattern of plain, non-volatile `get`s paired with strong `set`s can lead to correct behavior, given the class satisfies some constraints (which we did not have time to fully understand). Furthermore, the JMM states that writes/reads to/from references are *always atomic* [8] and (as per the JMM's definition of atomic) immediately visible to all other threads. The tail of the queue is a reference, so *the non-volatile read ends up being atomic by virtue of the memory model.* The developers likely opt to use a non-volatile read to prevent the JVM from adding any additional synchronization logic into the code.

This access pattern is widely used throughout Pekko. Throughout `Mailbox`, `Dispatcher`, and `Actor`, the developers mix non-volatile `get` operations on fields and higher strength `set` operations (both release and sequentially consistent).

The correctness of the queue is critical to the correctness of Pekko. The actor send rule is implemented by the happens-before relationship between enqueues and dequeues of messages in the `Mailbox`. The subsequent processing rule relies on that property as well as the fact that actors process messages serially once scheduled.

4 Actors Moving Between Threads

An important aspect of actors is that they are not thread-bound. To demonstrate the point, we created the example program to detect when an actor moves between threads. Our source code is publicly available here [14]. Figure 2 shows the overview of the two actors, `Yeller` and `Printer`, in our example. The `Yeller` "yells" when it moves between threads. It processes a message that contains the `id` of the thread it last executed on (or a sentinel value of 0 to start), comparing it with the `id` of the current thread. While the `id`s match, the `Yeller` sends itself a message (to process in the future) with the current thread `id`. Once the `id`s differ, the `Yeller` sends a string to display to the `Printer`.

The `Printer` demonstrates how an actor can protect a resource—namely, the standard output stream. The `Printer` takes a String as a message and then writes it to the standard output stream. As the sole actor responsible for displaying messages, `Printer` requires no internal concurrency primitives (besides receiving messages) to guard the standard output stream because it has exclusive access. If each `Yeller` attempted to write directly to the standard output stream, it could create a data race, and jumble several messages which were meant to be printed at once.

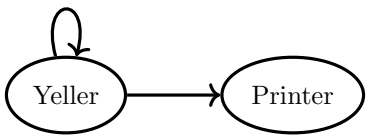

Figure 2: Example Behavior Model

Additionally, the `Yeller` actor is *almost* referentially transparent. It does not have a state of its own, as it uses a message to provide all necessary input data. However, the actor does make a nondeterministic call to get the thread `id`, which violates referential transparency. That itself is not a problem—the actor is designed this way. However, it is possible for the principle of actors moving between threads to break other concurrent mechanisms.

4.1 Breaking Reentrant Locks

Our intuition is that actors break the assumptions and guarantees made by other concurrency primitives in the JVM. If a developer who was familiar with the JMM was attempting to write a highly concurrent application using Pekko, they might do what they are familiar with, add a lock around critical sections of code. However, as mentioned, actors do not always behave as expected. To show this, we modified the `Yeller` actor described in the previous section into a new actor: `Deadlocker`. `Deadlocker` uses a closure to capture state—an exclusive instance of a `ReentrantLock`. A `ReentrantLock` is a lock which can be locked and unlocked multiple times given you are already the owner. Because the lock is actor-exclusive, there is no contention for it, and each `Deadlocker` actor should be able to reacquire the lock over and over.

However, when an actor moves between threads, the internal state of the lock becomes corrupt, as shown in Figure 3. Locks in Java are *thread-based*. The owner of the lock is based on the thread which issued the call to lock. As a result, once the actor moves between threads, it becomes locked out of its own *exclusive* resource, which is counterintuitive. Therefore, mixing paradigms breaks the assumptions made in both Pekko and the underlying JVM, resulting in possible data races and in this case, a deadlock.

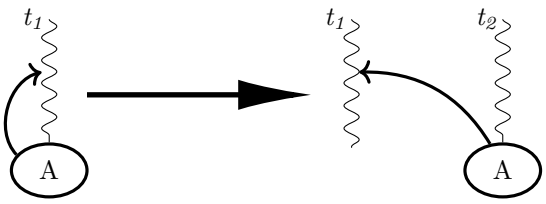

Figure 3: Reentrant Lock Example

5 Discussion

In theory, you could implement a *pure actor-based library as a Domain Specific Language (DSL)* with typed data structures. Control flow and primitives (e.g. memory cells) become primitive actors supplied by the library. Programmers can create new actors which can only message and/or compose actors. In other words, you cannot write arbitrary code for an actor to run, and thus we cannot capture data like a `ReentrantLock` from the example in Section 4.1. This can be enforced at compile time via type checking. Furthermore, the entirely DSL specification can include its own concurrency primitives—an actor which implements the logic of a semaphore (by sending continuations to execute once the semaphore has been acquired) is specified in [5].

6 Conclusion

The Actor model enables highly concurrent and safe programs. Actors communicate strictly via message passing and, when implemented at the language level, restrict or outright reject other forms of concurrency. Pekko is an actor library written in Scala, and uses the Java Memory Model to ensure correct concurrent performance.

The implementation of Pekko is difficult to follow due to: its highly configurable nature, deep levels of abstraction, and its use of mixed atomic memory orders. Often in the source code, release or sequentially consistent `set` operations occur on data fields which use non-volatile (similar to non-atomic) `get` operations. Reading the source code, it is not clear Pekko should guarantee correct performance as happens-before relationships should not be guaranteed given this mixture. However, additional reading about the Java Memory Model [11, 10] suggests that the implementation is correct and that Pekko is exploiting niche features of the memory model to improve performance.

The implementation of Pekko is correct, given that the programmer avoids mixing concurrent programming paradigms or very carefully configures for them. In an attempt to be ergonomic, Pekko allows actors to be defined via closures containing arbitrary code. As a result, it is left up to the programmer to avoid hazardous uses of the Java Memory Model that could break assumptions and guarantees provided by actors in Pekko. Section 4.1 demonstrated that an actor can fail to acquire an exclusive lock due to the nature of actor scheduling in Pekko.

The crux of the issue is that Pekko allows arbitrary code to govern an actor's behavior. It cannot eliminate actors which clearly use other concurrency primitives. It is our opinion that a strictly, purely actor library could be implemented as a domain specific language inside of Scala. However, many of the conveniences given by Pekko are largely due to larger community uptake, overall system performance, and configurability. Academically, the DSL could use type checking and, therefore, potentially have more capabilities to rout out nefarious actor behaviors to be eliminated at compile time.

References

[1] Gul Agha and Carl Hewitt. "Concurrent Programming Using Actors: Exploiting Large-Scale Parallelism". In: *Readings in Distributed Artificial Intelligence*. Elsevier, 1988, pp. 398–407.

[2] Apache Software Foundation. *Apache Pekko: A toolkit for building highly concurrent, distributed, and resilient message-driven applications for Java and Scala*. urlhttps://pekko.apache.org/. Version 1.3.0. Accessed: 2025-12-03. 2025.

[3] Ian F Currie. "NewSpeak: A Reliable Programming Language". In: *High-Integrity Software*. Springer, 1989, pp. 122–158.

[4] *Erlang*. https://www.erlang.org. Erlang. (Visited on 10/23/2025).

[5] Irene Greif. "Semantics of communicating parallel processes." eng. Accepted: 2010-08-31T13:57:01Z. Thesis. Massachusetts Institute of Technology, 1975. URL: `https://dspace.mit.edu/handle/1721.1/57710` (visited on 12/04/2025).

[6] Carl Hewitt et al. "Actor Induction and Meta-Evaluation". In: *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. New York, NY, USA: ACM, 1973, pp. 153–168. ISBN: 1-4503-7349-6.

[7] *Information technology – Programming languages – C*. ISO/IEC 9899:2011. International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC). 2011. URL: `https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf`.

[8] *Java Memory Model Spec*. https://docs.oracle.com/javase/specs/jls/se25/html/jls-17.html. (Visited on 12/05/2025).

[9] *Java Virtual Machine*. https://docs.oracle.com/en/java/javase/25/vm/java-virtual-machine-technology-overview.html. (Visited on 12/05/2025).

[10] Doug Lea. *JDK Memory Order Modes*. https://gee.cs.oswego.edu/dl/html/j9mm.html. Nov. 2018.

[11] Jeremy Manson, William Pugh, and Sarita V. Adve. "The Java Memory Model". In: *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '05. New York, NY, USA: Association for Computing Machinery, Jan. 2005, pp. 378–391. ISBN: 978-1-58113-830-6. DOI: 10.1145/1040305.1040336. (Visited on 11/27/2024).

[12] *Pekko and JMM*. https://pekko.apache.org/docs/pekko/current/general/jmm.html. Oct. 2025.

[13] *Scala*. https://www.scala-lang.org. (Visited on 12/05/2025).

[14] David Schwartz and Joeseph Wiseman. *CS 554 Final Project Code*. May 2025.

[15] *VarHandle – Java SE 22 API Documentation*. https://docs.oracle.com. Accessed: 2025-12-05. Oracle, 2025. URL: https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/lang/invoke/VarHandle.html.

[16] Dmitry Vyukov. *Non-intrusive MPSC node-based queue*. https://www.1024cores.net. Accessed: 2025-12-05. URL: https://www.1024cores.net/home/lock-free-algorithms/queues/non-intrusive-mpsc-node-based-queue.